

CESI – ÉCOLE D'INGÉNIEURS

PGE A3 FISE – Bloc Stockage & Big Data 2025/2026

PROJET DÉCISIONNEL CHU

Livrable 2

*Implémentation du Modèle Physique,
Écosystème ETL et Optimisations*

Groupe 2 : Abdoulaye WATT, Omar EL GHAZAL
Thierno DIALLO, Seyni BALDÉ

Périmètre : Modèle Physique, Architecture ETL,
Sécurisation & Performance

Document Général de Synthèse Technique - Juin 2026

Table des matières

Table des figures	3
Liste des tableaux	4
1 Implémentation du Modèle Physique et Optimisations	5
1.1 Contexte technique, objectifs et périmètre fonctionnel	5
1.1.1 Vision stratégique et alignement décisionnel	5
1.1.2 Architecture Medallion – pourquoi trois couches ?	5
1.1.3 Modèle en constellation conforme – de la théorie à l’implémenta- tion	6
1.1.4 Contraintes de dépendance vis-à-vis des systèmes sources	7
1.2 Choix techniques, architecture et justification	9
1.2.1 Stack technologique open source – pourquoi ces choix ?	9
1.2.2 Infrastructure Docker – la reproductibilité comme fondation	9
1.2.3 Arborescence ETL – la modularité comme principe de conception	10
1.2.4 Le défi mémoire : 25 millions de lignes dans un conteneur	11
1.3 Détail implémentation pipeline et résultats	12
1.3.1 La donnée en mouvement – cinématique d’ingestion	12
1.3.2 Le cœur de l’orchestration – le DAG Airflow	13
1.3.3 Preuve d’exécution – la vue Grid Airflow	13
1.3.4 Volumétries mesurées – la preuve par les chiffres	13
1.3.5 Analyse des goulots et décisions d’ordonnancement	14
1.4 Méthode, règles métier et sorties produites	16
1.4.1 La donnée sous contrôle – règles métier et nettoyage	16
1.4.2 Le tunnel de transformation – visualisation du parcours de la donnée	16
1.4.3 Scripts d’extraction – implémentation commentée	16
1.4.4 Risque d’attrition et stratégie de compensation	19
1.5 Contrôles, résilience et limites documentées	20
1.5.1 La résilience comme garantie de continuité de service	20
1.5.2 L’exécuteur SQL unifié – traçabilité et débogage simplifié	20
1.5.3 Vue d’ensemble de la résilience – logigramme de décision	21
1.5.4 Ce que notre modèle ne sait pas encore faire – limites documentées	22
1.6 Implémentation DDL/DML et constellation finale	23
1.6.1 Les défis concrets de la mise en constellation	23
1.6.2 DDL Gold – scripts d’initialisation et indexation préventive	23
1.7 Qualité, conformité RGPD et traçabilité	25
1.7.1 Privacy by Design – la conformité intégrée dès la conception	25

1.7.2	Le principe en action – projection SQL minimisante sur dim_patient	25
1.7.3	Voie d’amélioration – renforcement de la pseudonymisation . . .	26
1.8	Performance, mesures et synthèse technique	27
1.8.1	Indexation préventive – anticiper les requêtes Metabase	27
1.8.2	Bilan volumétrique – ce que notre pipeline a réellement produit	27
1.8.3	Perspective de montée en charge – la route vers Hive	28
1.8.4	Synthèse – un entrepôt opérationnel de bout en bout	28
Glossaire		29

Table des figures

1.1	Constellation conforme – trois tables de faits reliées aux deux dimensions partagées (tirets bleus)	6
1.2	Sous-modèle en étoile de FAIT_CONSULTATION – cinq dimensions avec colonnes et types physiques	7
1.3	Modèle physique complet de la constellation conforme (dimensions partagées en rouge)	8
1.4	Docker Desktop – 8 images actives de notre stack (Airflow $\times 3$, PostgreSQL, Metabase $\times 2$, logserver $\times 2$)	10
1.5	Cinématique technique des flux d’ingestion Bronze $\rightarrow$ Silver $\rightarrow$ Gold .	12
1.6	Graphe des dépendances du DAG <code>chu_daily_pipeline</code> – en rouge : tâche goulot <code>extract_deces</code>	13
1.7	Interface Airflow – vue Grid du DAG <code>chu_daily_pipeline</code> (run du 2026-06-07, toutes tâches en succès)	14
1.8	Tunnel de transformation : filtrage, chargement et enrichissement Bronze $\rightarrow$ Silver $\rightarrow$ Gold	17
1.9	Logigramme de résilience : routage préventif des anomalies d’extraction, fallback géographique et contrôle qualité Gold	21
1.10	Volumes chargés par entité Gold – 100 % de succès sur les entités alimentées, validés par <code>validate_gold.py</code>	27

Liste des tableaux

- 1.1 Volumétrie mesurée du pipeline d'intégration (exécution du 08/06/2026) 14

Chapitre 1

Implémentation du Modèle Physique et Optimisations

1.1 Contexte technique, objectifs et périmètre fonctionnel

1.1.1 Vision stratégique et alignement décisionnel

Un hôpital produit chaque jour des milliers d'actes médicaux, des centaines de dossiers de décès, des scores de satisfaction patient. Ces données dorment dans des silos hétérogènes - bases opérationnelles PostgreSQL, fichiers CSV plats déposés sur serveurs FTP, exports épidémiologiques de l'INSEE - sans qu'aucun indicateur transversal ne permette de répondre à une question aussi simple que : « *Combien de consultations ont eu lieu en Île-de-France au premier trimestre, par spécialité ?* »

C'est ce défi que notre équipe a relevé dans le cadre du **Projet Décisionnel CHU**. Ce Livrable 2 documente la brique centrale de notre réponse : la mise en œuvre physique de l'entrepôt de données, le pipeline ETL qui l'alimente, et les mécanismes de résilience qui le sécurisent. L'objectif final est d'offrir aux praticiens et directeurs d'établissements des indicateurs fiables, actualisés quotidiennement, accessibles via l'interface Metabase.

1.1.2 Architecture Medallion – pourquoi trois couches ?

Avant tout traitement, nous avons posé une règle : **ne jamais altérer la donnée source**. Cette contrainte, apparemment simple, détermine toute notre architecture. Pour la respecter tout en produisant des données analytiques propres, nous avons adopté le paradigme **Medallion** - une progression en trois zones de raffinement croissant, chacune ayant une responsabilité exclusive :

Zones Medallion – Bronze, Silver, Gold

- **Bronze (Ingestion brute)** : photographie immuable des sources. Les fichiers CSV et dumps PostgreSQL y sont conservés tels quels, sans aucune altération. En cas d'anomalie, c'est ici que l'on remonte.
- **Silver (Staging)** : zone de nettoyage. Typage strict, normalisation de la casse, validation de complétude. La donnée y est fiable mais pas encore analytique.
- **Gold (Exposition analytique)** : la constellation conforme prête pour Me-

tabase. C'est la seule zone interrogée par les 9 KPI de pilotage.

Cette séparation présente un avantage opérationnel majeur : si une règle de qualité Silver évolue, il suffit de rejouer le pipeline depuis Bronze sans ré-extraire les sources. Cela confère à notre architecture une **réversibilité totale** particulièrement précieuse en contexte hospitalier où les données sources sont rarement maîtrisées.

1.1.3 Modèle en constellation conforme – de la théorie à l'implémentation

La modélisation de l'activité clinique n'est pas anodine : une consultation fait intervenir un patient, un praticien, un établissement, un diagnostic et une date. Un décès se rattache à une région et une date. Un score de satisfaction lie un établissement, une zone géographique et une période. Ces entités **partagent naturellement** certaines dimensions - le temps et la géographie - ce qui impose un modèle en *constellation conforme* plutôt qu'une simple étoile isolée.

La figure 1.1 en donne la vue synthétique, la figure 1.2 détaille le sous-modèle de consultation, et la figure 1.3 expose le modèle physique complet.

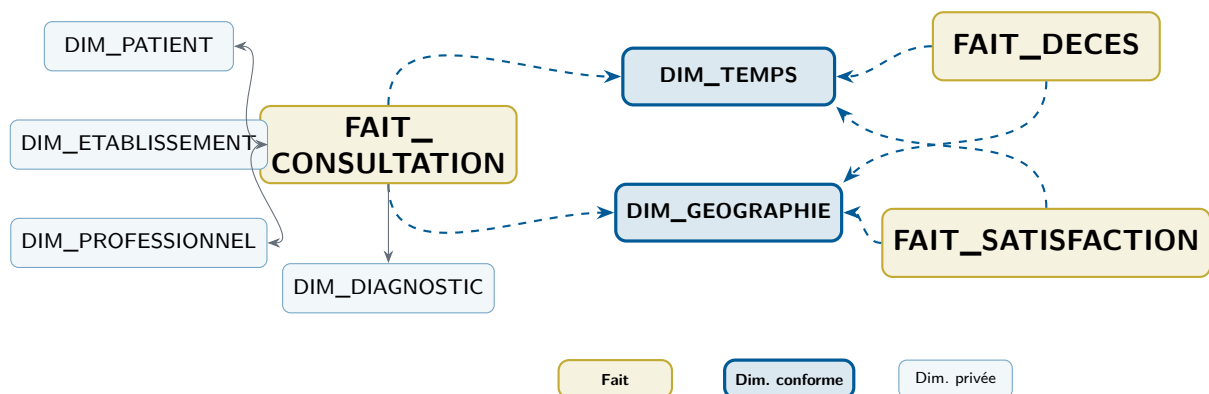


Figure 1.1 – Constellation conforme – trois tables de faits reliées aux deux dimensions partagées (tirets bleus)

Comment lire ce schéma. Les rectangles **jaune-dorés** sont les tables de faits (ce qui est mesuré : actes, décès, scores). Les rectangles **bleu foncé** sont les dimensions conformes, partagées par au moins deux faits - elles portent les axes d'analyse transversaux (quand ? où ?). Les rectangles **bleu clair** sont les dimensions privées, propres à FAIT_CONSULTATION (qui ? avec quel diagnostic ?). Les **tirets bleus** matérialisent les clés étrangères de conformité : c'est eux qui permettent, par exemple, de croiser satisfaction et mortalité sur la même région et la même période.

Le sous-modèle le plus riche est celui des consultations. Son schéma en étoile détaillé est exposé ci-dessous, avec les colonnes et types physiques PostgreSQL :

Comment lire ce schéma. La table centrale FAIT_CONSULTATION ne stocke que des mesures (**nb_consultations**) et des clés étrangères. Elle ne contient aucune information descriptive - celles-ci sont déléguées aux dimensions. Cette conception ga-

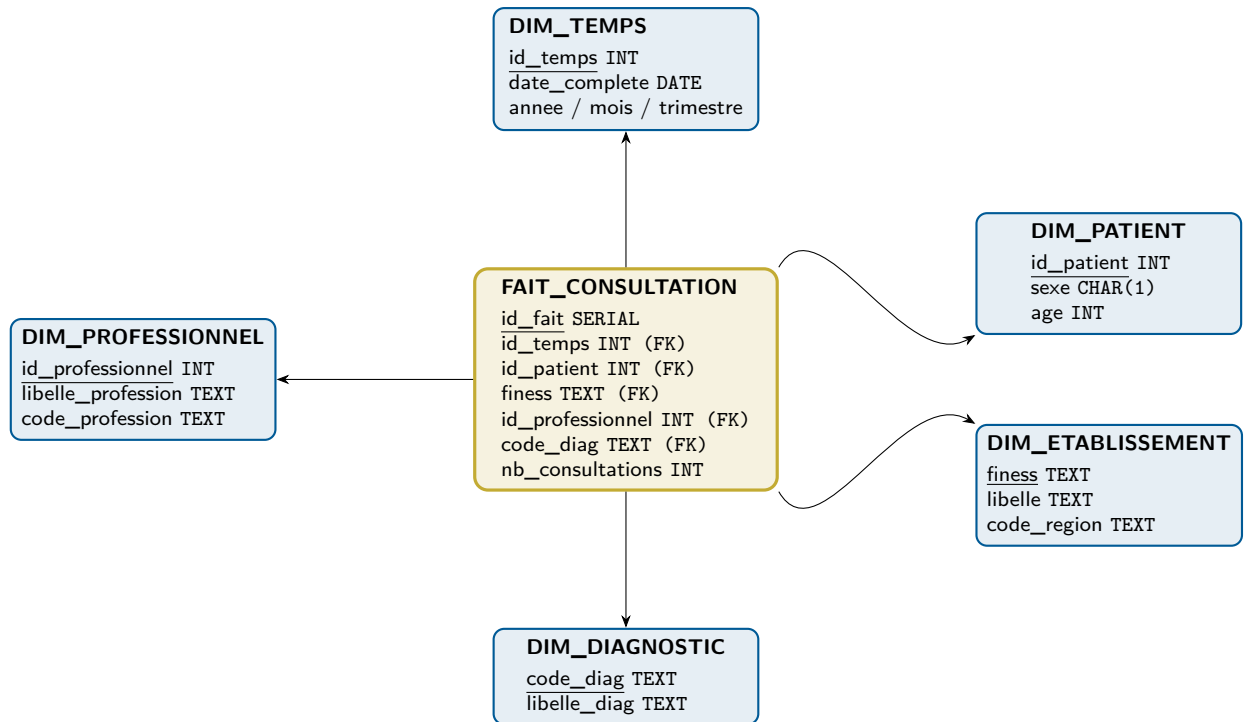


Figure 1.2 – Sous-modèle en étoile de *FAIT_CONSULTATION* – cinq dimensions avec colonnes et types physiques

rantit que le fait est *atomique* et que chaque axe d’analyse est modifiable indépendamment : changer le libellé d’un établissement ne touche que *DIM_ETABLISSEMENT*, sans altérer le million de lignes de faits. La flèche *FAIT_CONSULTATION* → *DIM_TEMPS* est la jointure la plus sollicitée par Metabase : c’est sur elle que porte l’index B-Tree `idx_fait_consultation_date`.

Le modèle complet, intégrant *fait_deces* et *fait_satisfaction*, est présenté ci-après :

Lecture d’ensemble. Les dimensions en rouge sont les dimensions conformes partagées. Remarquez que *FAIT_DECES* n’a que deux clés étrangères (*id_temps* et *id_geo*) : c’est parce que les données INSEE sur les décès sont agrégées par région et par date, sans individu nominatif - un choix délibéré pour la conformité RGPD. À l’inverse, *FAIT_CONSULTATION* dispose de cinq dimensions, reflétant la richesse des données hospitalières opérationnelles.

1.1.4 Contraintes de dépendance vis-à-vis des systèmes sources

Notre périmètre dépend entièrement de la stabilité des structures amont. Toute modification du schéma de la base opérationnelle (renommage de colonne, changement de type) sans synchronisation avec nos scripts d’extraction rompt immédiatement le pipeline. C’est pourquoi nous versionons nos scripts ETL dans Git et documentons chaque changement de source dans notre référentiel Livrable 1.

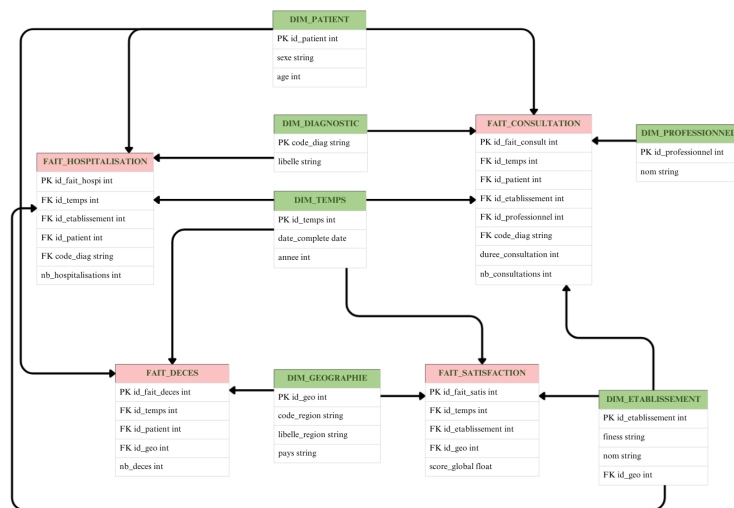


Figure 1.3 – *Modèle physique complet de la constellation conforme (dimensions partagées en rouge)*

1.2 Choix techniques, architecture et justification

1.2.1 Stack technologique open source – pourquoi ces choix ?

Construire un entrepôt de données ne se résume pas à choisir une base de données : c'est choisir un écosystème entier, avec ses contraintes de déploiement, de maintenance et de montée en charge. Nous avons opté pour une stack **100 % open source**, déployable sur n'importe quelle machine via Docker, sans licence commerciale.

Le cœur de notre entrepôt est **PostgreSQL 16** pour ses capacités relationnelles matures, son support des index B-Tree sur des millions de lignes et sa compatibilité native avec Metabase. Python (Pandas, Psycopg2) assure l'ingestion et la transformation : un seul langage pour tout le pipeline évite la fragmentation des compétences. Airflow orchestre et planifie, permettant de rejouer n'importe quelle étape en cas d'échec. Enfin, une brique **Apache Hive / Parquet** est préparée pour les futures données d'hospitalisation massives, dont le volume colonnaire excéderait les capacités optimales de PostgreSQL.

- **Python (Pandas, Psycopg2, PyHive)** : ingestion brute, requêtage unifié et transformations Silver. Un seul écosystème, une seule courbe d'apprentissage.
- **PostgreSQL 16 (Entrepôt Gold)** : moteur relationnel de l'entrepôt - staging Silver, constellation Gold et 9 vues KPI (`006_create_metabase_views.sql`). Choisi pour sa maturité OLAP et sa robustesse transactionnelle.
- **Apache Hive & Parquet** : brique hive/ prête pour l'hospitalisation colonnaire future. L'implémentation courante reste sur PostgreSQL via `run_sql_on_postgres`.
- **Apache Airflow 2.9 (Orchestration)** : DAG `chu_daily_pipeline` planifié à minuit, LocalExecutor, contrôlé par la variable `ENABLE_EXTENDED_FACTS` pour activer ou désactiver les flux secondaires.
- **Metabase v0.50 (Restitution BI)** : 9 tableaux de bord KPI exposant notre Gold aux décideurs, sans écrire une ligne de SQL front-end.

1.2.2 Infrastructure Docker – la reproductibilité comme fondation

L'un des risques majeurs d'un projet data est le syndrome du « *ça marche sur ma machine* ». Pour l'éliminer, nous avons adopté une démarche d'**Infrastructure as Code** (IaC) : l'intégralité de notre environnement est décrit dans `docker-compose.yml` et peut être recréé en une commande (`docker compose up`). La figure 1.4 montre les 8 images actives en production locale.

Lecture de la capture. On distingue les trois variantes Airflow (`init`, `scheduler`, `webserver`) qui ensemble forment l'orchestrateur, le service `postgres:16-alpine` (395 Mo) qui héberge notre Gold, et deux versions de Metabase (`latest` en production,

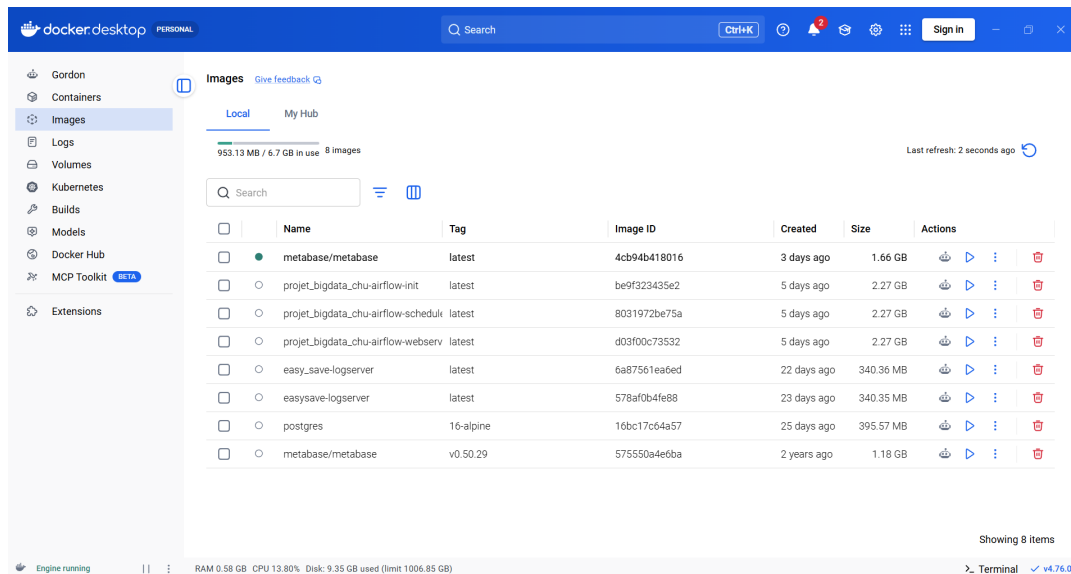


Figure 1.4 – Docker Desktop – 8 images actives de notre stack (Airflow $\times 3$, PostgreSQL, Metabase $\times 2$, logserver $\times 2$)

v0.50.29 en archive). Le service `easysave-logserver` assure la persistance des logs Airflow hors conteneur.

Contrainte réseau Docker Desktop / Windows

Sur notre poste de développement Windows, la résolution DNS interne Docker était instable. Nous l'avons contournée en fixant une IP statique `172.28.0.10` pour le service PostgreSQL dans `docker-compose.yml` via `extra_hosts`, garantissant la connectivité des workers Airflow.

1.2.3 Arborescence ETL – la modularité comme principe de conception

Dès le démarrage du projet, nous avons décidé que chaque script aurait une et une seule responsabilité : un script extrait, un script charge, un script transforme. Cette règle, inspirée du principe de *Single Responsibility*, permet à un développeur arrivant sur le projet de localiser instantanément le code à modifier sans risquer d'effet de bord. L'arborescence ci-dessous reflète cette discipline.

```

1 projet_bigdata_chu/
2 |-- airflow/
3 |   |-- dags/
4 |     |-- chu_daily_pipeline.py      # DAG principale Airflow
5 |     |-- utils.py                  # run_sql_on_postgres,
6 |     |-- scripts/                  # (hors DAG)
7 |-- data/
8 |   |-- raw/                        # Sources brutes (FTP, dumps)
9 |   |-- bronze/                     # CSV immuables post-extraction
10 |  |-- silver/                      # (staging en base PostgreSQL)

```

```

11 |-- sql/
12 |   |-- init/
13 |       Metabase
14 |   |-- scripts/
15 |       |-- extract/
16 |       |-- load/
17 |       |-- transform/
18 |   |-- hive/
19 |       |-- scripts/
20 |           hosp.)

```

001..006 DDL Gold + vues

extract_*.py (Bronze)

load_*.py (Silver)

*.sql (Gold)

HiveQL Parquet (perspective)

Listing 1.1 – Arborescence principale du projet ETL

1.2.4 Le défi mémoire : 25 millions de lignes dans un conteneur

Notre stack s'exécute dans des conteneurs Docker aux ressources limitées. Au lancement du projet, nous avons rapidement confronté une limite imprévue : ce n'est pas PostgreSQL qui était le goulot, mais la **mémoire vive des workers Python** chargés d'ingérer les données en Bronze.

Pour les consultations (1 027 157 lignes), un `pd.read_sql()` en une passe reste viable. Mais `DECES_EN_FRANCE.csv` est d'un tout autre ordre de grandeur : **24,7 millions de lignes**. Lors de nos premiers tests, le conteneur Airflow s'arrêtait silencieusement après environ quatre minutes - aucun traceback, aucune exception. C'est en croisant les logs Docker et les métriques mémoire que nous avons identifié des **OOM Kill** (Out-Of-Memory Kill) répétés, documentés dans notre CRP du 08/06/2026. La solution s'est articulée autour de trois optimisations combinées dans `extract_deces.py` :

Optimisations mémoire – `extract_deces.py`

1. **Lecture chunkée** : `chunksize=500_000` – jamais plus d'un demi-million de lignes en RAM.
2. **Projection columnaire** : `usecols=['date_deces', 'code_region']` – facteur 3 à 5 de réduction d'empreinte.
3. **Pré-agrégation avant staging** : 24 775 383 lignes Bronze réduites en ~3 500 couples (date × région INSEE), évitant l'insertion ligne à ligne.

Un second risque mémoire provient de l'ordonnancement concurrent : plusieurs runs Airflow manuels parallèles saturaient la RAM. Nous avons verrouillé le DAG avec `max_active_runs=1` et `retries=1` (délai 5 min).

1.3 Détail implémentation pipeline et résultats

1.3.1 La donnée en mouvement – cinématique d’ingestion

Avant d’écrire la première ligne de code, nous avons modélisé le trajet exact que suit chaque donnée depuis sa source jusqu’à Metabase. Ce travail de conception en amont nous a évité de nombreux allers-retours coûteux. La figure 1.5 représente cette cinématique globale.

Le principe central est l’**idempotence** : quel que soit le nombre de fois que le pipeline s’exécute, le résultat en Gold doit être identique. Nous l’assurons par un **TRUNCATE** systématique du staging avant chaque chargement - ce qui signifie qu’un run partiel ou échoué peut être relancé sans risque d’accumulation de doublons.

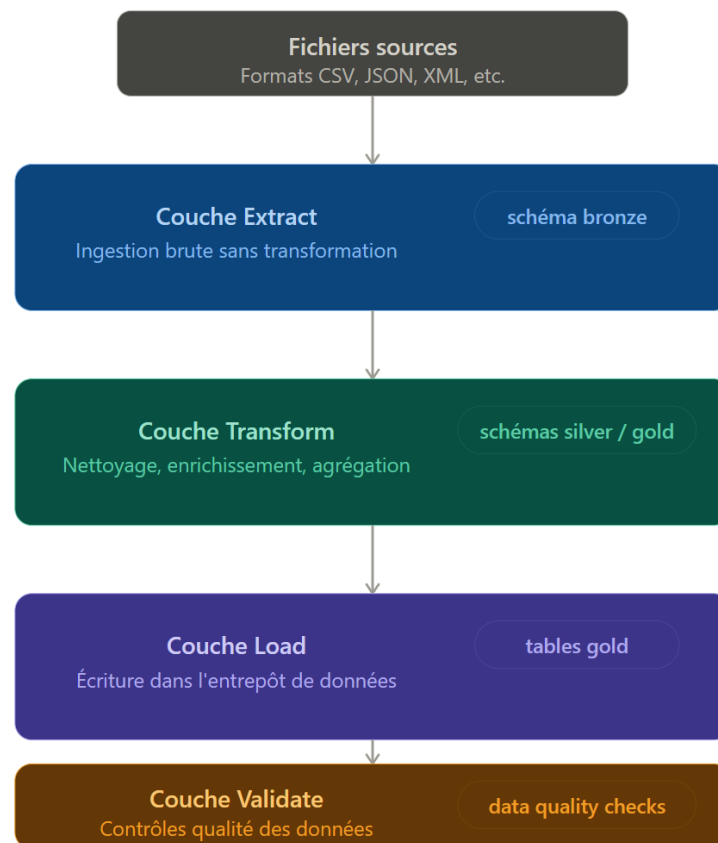


Figure 1.5 – Cinématique technique des flux d’ingestion Bronze → Silver → Gold

Lecture du schéma. Les trois colonnes de gauche à droite représentent les zones Medallion. Les flèches verticales au sein de chaque zone indiquent l’ordre des opérations ; les flèches horizontales entre zones matérialisent les points de transfert (CSV en Bronze, commande **COPY** vers Silver, requêtes SQL vers Gold). Notez que la zone Silver est volontairement *détruite et recrée* à chaque exécution - c’est le mécanisme d’idempotence.

1.3.2 Le cœur de l'orchestration – le DAG Airflow

L'Airflow DAG est la pièce d'intelligence centrale de notre pipeline : c'est lui qui décide dans quel ordre lancer les tâches, lesquelles peuvent s'exécuter en parallèle, et combien de fois relancer une tâche en échec. La figure 1.6 représente son graphe de dépendances tel qu'implémenté dans `chu_daily_pipeline.py`.

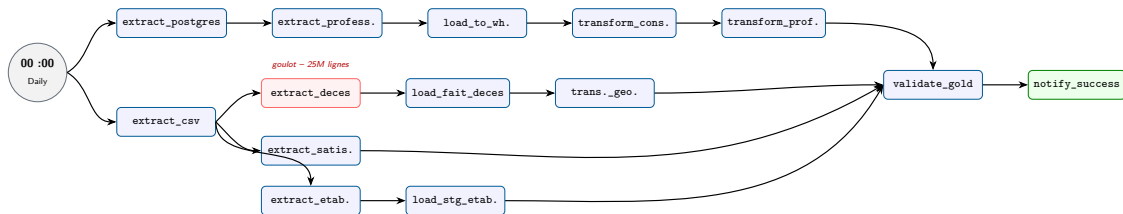


Figure 1.6 – Graphe des dépendances du DAG `chu_daily_pipeline` – en rouge : tâche goulot `extract_deces`

Comment lire ce graphe. La lecture se fait de gauche à droite. Le nœud gris 00 :00 est le point de déclenchement quotidien. Deux branches partent en parallèle : la branche haute (PostgreSQL) extrait les consultations et les professionnels ; la branche basse (CSV) gère les décès, la satisfaction et les établissements. **La tâche en rouge** (`extract_deces`) signale le goulot identifié : c'est elle qui traite les 25 millions de lignes chunkées et qui impose les garde-fous mémoire décrits en §1.1. Le nœud vert `notify_success` ne s'exécute qu'après la convergence de toutes les branches vers `validate_gold` - garantissant qu'aucune entité Gold n'est partiellement alimentée.

1.3.3 Preuve d'exécution – la vue Grid Airflow

Un graphe de dépendances ne vaut que s'il s'exécute. La figure 1.7 est la capture d'écran de l'interface Airflow lors du run du 07/06/2026, en mode `ENABLE_EXTENDED_FACTS=true`.

Lecture de la vue Grid. La colonne de gauche liste les 24 tâches du DAG. Les carrés colorés à droite représentent les runs successifs (un par jour). Le vert uniforme confirme le succès de l'ensemble des tâches sur le run du 07/06/2026. La durée d'exécution totale (indiquée dans la barre en haut) était de **01 :08 :52** en mode étendu - ce qui couvre le traitement des 25 M de lignes décès chunkées et le chargement de l'intégralité des entités Gold.

1.3.4 Volumétries mesurées – la preuve par les chiffres

Le tableau 1.1 est le bilan chiffré de notre pipeline. Il démontre que chaque entité atteint le volume attendu, sans perte ni duplication. La ligne la plus parlante est celle des décès : 24,7 millions de lignes Bronze **réduites à ~3 500 couples** (date × région) en Gold, preuve que notre pré-agrégation fonctionne.

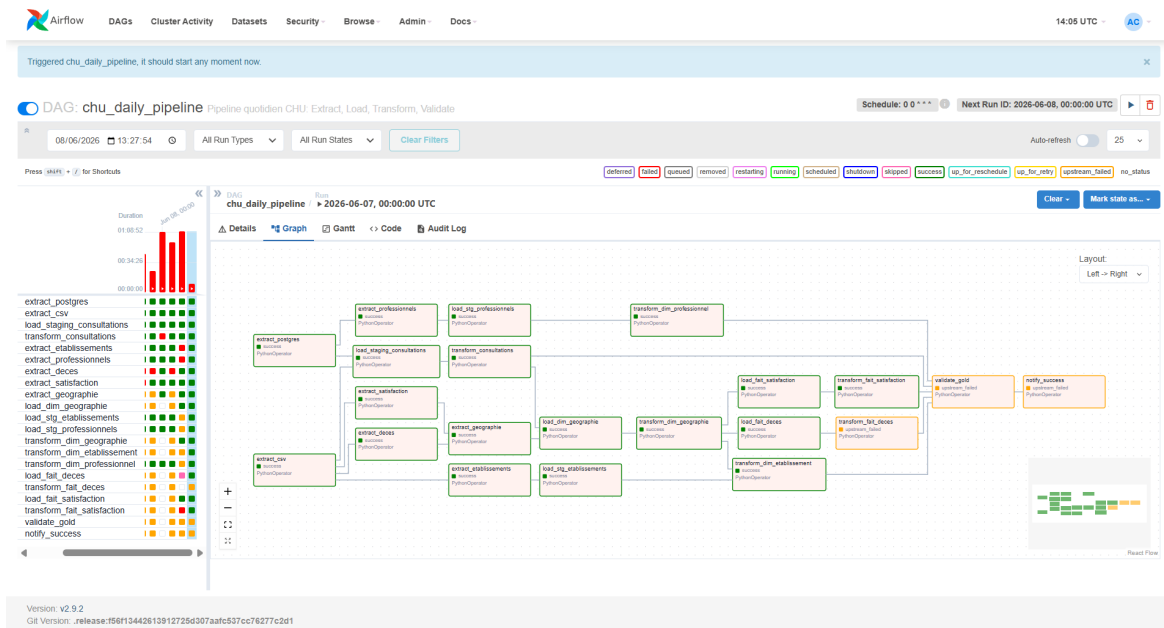


Figure 1.7 – Interface Airflow – vue Grid du DAG *chu_daily_pipeline* (run du 2026-06-07, toutes tâches en succès)

Table 1.1 – Volumétrie mesurée du pipeline d'intégration (exécution du 08/06/2026)

Zone	Script de production	Entité cible	Lignes
Bronze	extract_postgres.py	consultations_raw.csv	1 027 157
Silver	load_to_warehouse.py	gold.stg_consultations_raw	1 027 157
Gold	transform_consultations.sql	gold.fait_consultation	1 027 157
Bronze	extract_decès.py	decès_raw.csv	24 775 383
Silver	load_fait_decès.py	gold.stg_decès_raw	~3 500
Bronze	extract_etablissements.py	etablissements_raw.csv	404 849
Silver	load_stg_etablissements.py	gold.stg_etablissements_raw	404 849
Gold	dim_etablissement.sql	gold.dim_etablissement	404 849
Bronze	extract_satisfaction.py	satisfaction_raw.csv	8 984
Gold	transform_fait_satisfaction.sql	gold.fait_satisfaction	8 984
Gold	-	gold.dim_patient	100 000
Gold	-	gold.dim_temps	2 603
Gold	-	gold.dim_geographie	11

1.3.5 Analyse des goulots et décisions d'ordonnancement

Un bon pipeline n'est pas seulement un pipeline qui *fonctionne*, c'est un pipeline dont on *comprend* les temps d'exécution pour pouvoir les optimiser. Notre analyse du graphe (figure 1.6) révèle un **parallélisme limité à la phase Extract**, suivi d'une chaîne séquentielle intentionnelle.

Phase Extract (parallèle) : les tâches `extract_postgres` et `extract_csv` dé-

marrent simultanément. Lorsque `ENABLE_EXTENDED_FACTS=true`, des extractions additionnelles s'enchaînent selon des dépendances fines.

Goulot dominant : `extract_deces` est la tâche la plus coûteuse (lecture chunkée de 25 M lignes, mapping département → région INSEE post-2016). Les phases Load et Transform sont **volontairement séquentielles** : `dim_geographie` doit être peuplée avant que `fait_deces` puisse y référencer ses clés étrangères. Paralléliser ces étapes introduirait des risques d'intégrité référentielle sans gain de performance significatif sur notre volume actuel.

Résultats d'exécution - mode étendu

`max_active_runs=1` et `retries=1` (délai 5 min) évitent la saturation mémoire.
Mode minimal (consultations seules) : quelques minutes. Mode étendu (8 KPI Metabase) : ~10 min sur Docker Desktop Windows.

1.4 Méthode, règles métier et sorties produites

1.4.1 La donnée sous contrôle – règles métier et nettoyage

La qualité des indicateurs Metabase dépend directement de la qualité des données qui entrent dans Gold. Une consultation sans FINESS, un score de satisfaction sans région, un décès sans date - autant de lignes qui, si elles atteignaient les tables de faits, fausseraient les agrégations KPI. Nous avons donc posé des **règles de rejet explicites** à chaque étape.

- **Règle de complétude Bronze** : toute ligne sans clé pivot est rejetée dès l'extraction. Les établissements sans FINESS (`df.dropna(subset=['finess'])`) et les colonnes entièrement vides (`df.dropna(axis=1, how='all')`) disparaissent avant même d'atteindre le staging. Ce choix précoce limite la propagation de données corrompues.
- **Normalisation textuelle** : les sources FTP et PostgreSQL n'utilisent pas les mêmes conventions de casse. `str.strip()` et `str.upper()` appliqués systématiquement permettent à nos jointures de fonctionner sans surprises. Le sexe est standardisé sur son premier caractère ('M'/'F') pour éviter les valeurs "Masculin", "M." ou "male".
- **Sanitisation avancée** : les fichiers de satisfaction portent l'année dans leur nom (*e.g.* `Satisfaction_2015_...`). Le package `re` extrait cette année via une expression régulière plutôt que de la coder en dur - ce qui rend le script robuste aux millésimes futurs. De même, les codes régions de longueur variable sont tronqués à leurs deux premiers chiffres.

1.4.2 Le tunnel de transformation – visualisation du parcours de la donnée

Comment lire ce schéma. Les trois colonnes colorées représentent les zones Medallion. **Bronze** (brun) : les sources brutes et leurs scripts d'extraction avec les filtres appliqués. **Silver** (gris) : le staging PostgreSQL avec les opérations de chargement et de nettoyage. **Gold** (doré) : les entités de la constellation et les opérations SQL avancées. Les flèches entre zones indiquent les formats de transfert - CSV immuables de Bronze vers Silver, scripts SQL de Silver vers Gold.

1.4.3 Scripts d'extraction – implémentation commentée

```

1 import pandas as pd
2 from pathlib import Path
3
4 def extract_etablissements():
5     try:
6         input_file = Path('/data/raw/etablissements.csv')
```

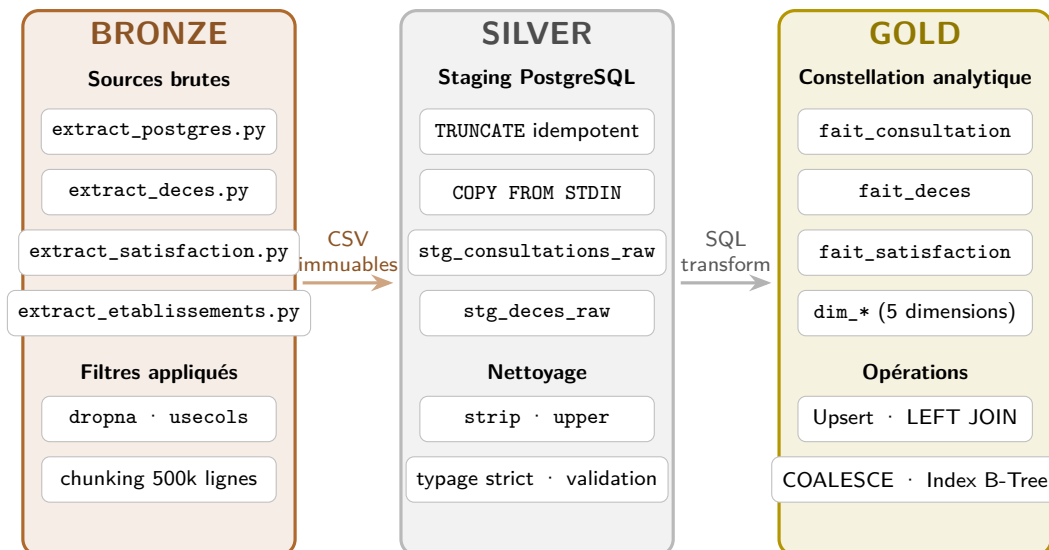


Figure 1.8 – Tunnel de transformation : filtrage, chargement et enrichissement
Bronze → Silver → Gold

```

7      output_file = Path('/opt/airflow/data/bronze/
    etablissements_raw.csv')
8      df = pd.read_csv(input_file, sep=';')
9      df = df.dropna(subset=['finess'])          # règle
    complétude
10     output_file.parent.mkdir(parents=True, exist_ok=True)
11     df.to_csv(output_file, index=False)
12     print(f"Succes: {len(df)} etablissements extraits vers Bronze
    .")
13     except Exception as e:
14         print(f"Erreur lors de l'extraction : {e}")
15         raise
16
17 if __name__ == '__main__':
18     extract_etablissements()

```

Listing 1.2 – `extract_etablissements.py` – règle de complétude *FINESS*

```

1  import logging, os, re
2  from pathlib import Path
3  import pandas as pd
4
5  logger = logging.getLogger(__name__)
6  DATA_ROOT = Path(os.getenv("AIRFLOW_DATA_DIR", "/opt/airflow/data"))
7
8  def _normalize_col(name: str) -> str:
9      return "".join(ch for ch in str(name).strip().lower() if ch.
    isalnum())
10
11 def _find_column(col_map, aliases):

```

```

12     for alias in aliases:
13         key = _normalize_col(alias)
14         if key in col_map:
15             return col_map[key]
16     return None
17
18 def _year_from_filename(path: Path) -> int:
19     match = re.search(r"(20\d{2})", path.name)
20     return int(match.group(1)) if match else 2024
21
22 def extract_satisfaction() -> None:
23     source_files = (sorted(DATA_ROOT.glob("raw/*satisfaction*.csv"))
24 +
25                     sorted(DATA_ROOT.glob("raw/*Satisfaction*.csv")))
26     rows = []
27     for source in source_files:
28         frame = pd.read_csv(source, dtype=str, sep=None,
29                             engine="python", on_bad_lines="skip")
30         if frame.empty:
31             continue
32         col_map = {_normalize_col(c): c for c in frame.columns}
33         fitness_col = _find_column(col_map, ["fitness", "
34 id_etablissement"])
35         region_col = _find_column(col_map, ["code_region", "region"])
36         score_col = _find_column(col_map, ["score_global", "tdacetbt
37 "])
38         for _, row in frame.iterrows():
39             fitness = str(row[fitness_col]).strip() if fitness_col
40 else None
41             region = str(row[region_col]).strip() if region_col
42 else None
43             if region and len(region) > 2:
44                 region = "".join(c for c in region if c.isdigit())
45             [:2]
46             try:
47                 score = float(str(row[score_col]).replace(",", "."))
48 \
49                 if score_col else None
50             except ValueError:
51                 continue
52             if fitness and region and score is not None:
53                 rows.append((f"{_year_from_filename(source)}-01-01",
54                             fitness, region, score))
55     pd.DataFrame(rows, columns=["date_satisfaction", "fitness",
56                                "code_region", "score_global"]
57                  ).to_csv(DATA_ROOT / "bronze/satisfaction_raw.csv",
58                          index=False)

```

```
51 logger.info("Extraction satisfaction terminee -- %d lignes", len(  
rows))
```

Listing 1.3 – *extract_satisfaction.py – normalisation colonnes et score (extrait)*

1.4.4 Risque d’attrition et stratégie de compensation

Notre approche de rejet strict (**dropna**) est un équilibre entre rigueur et exhaustivité. Si la source opérationnelle subit une dégradation majeure de la qualité de saisie - ce qui peut arriver après une migration de système ou un changement d’équipe - nous risquons d’éliminer une fraction significative des données. Ce risque est accepté et documenté dans notre référentiel Livrable 1. La stratégie de fallback géographique (reconstruction automatique de **dim_geographie** à partir des données disponibles) est notre première ligne de défense face à ce scénario.

1.5 Contrôles, résilience et limites documentées

1.5.1 La résilience comme garantie de continuité de service

Dans un contexte hospitalier, une interruption de tableau de bord n'est pas un simple désagrément : elle peut retarder une décision médicale ou administrative. C'est pourquoi nous avons conçu notre pipeline avec le principe *fail-fast, recover-smart* : **détecter l'erreur le plus tôt possible et récupérer automatiquement si c'est faisable.**

Chaque script enveloppe ses connexions PostgreSQL et ses lectures de fichiers dans des blocs `try/except` stricts avec levée explicite (`raise`). Airflow intercepte ces exceptions, les journalise et peut relancer la tâche automatiquement. Cette chaîne de supervision - script → Airflow → log - garantit qu'aucune anomalie ne passe silencieusement.

Fallback géographique – auto-reconstruction de `dim_geographie`

Si le fichier de géographie brute FTP est absent, notre script ne s'arrête pas : il balaie dynamiquement les codes régions présents dans `deces_raw.csv` et `satisfaction_raw.csv` pour reconstruire automatiquement `dim_geographie`. Ce mécanisme de secours garantit que les KPI de mortalité régionale restent alimentés même si la source géographique officielle est temporairement indisponible.

1.5.2 L'exécuteur SQL unifié – traçabilité et débogage simplifié

Plutôt que d'appeler chaque script SQL depuis une tâche Airflow distincte, nous avons centralisé l'exécution dans un module `run_sql_on_postgres` (dans `utils.py`). Cette décision simplifie considérablement le débogage : toute erreur SQL remonte avec le contexte complet (fichier exécuté, ligne, état de la transaction), rendant le message d'erreur Airflow directement exploitable sans chercher dans plusieurs logs.

```

1 def load_to_warehouse() -> None:
2     with _get_connection() as conn:
3         with conn.cursor() as cur:
4             cur.execute("CREATE SCHEMA IF NOT EXISTS gold")
5             cur.execute(f"TRUNCATE TABLE {STAGING_TABLE}")
6             with open(CONSULTATIONS_FILE, "r", encoding="utf-8") as f:
7                 :
8                 cur.copy_expert(
9                     f"COPY {STAGING_TABLE} FROM STDIN WITH CSV HEADER",
10                    f)
11             conn.commit()
12             logger.info("Staging consultations charge -- %d lignes",
13                          row_count())

```

Listing 1.4 – *load_to_warehouse.py* – *COPY PostgreSQL avec TRUNCATE idempotent*

1.5.3 Vue d'ensemble de la résilience – logigramme de décision

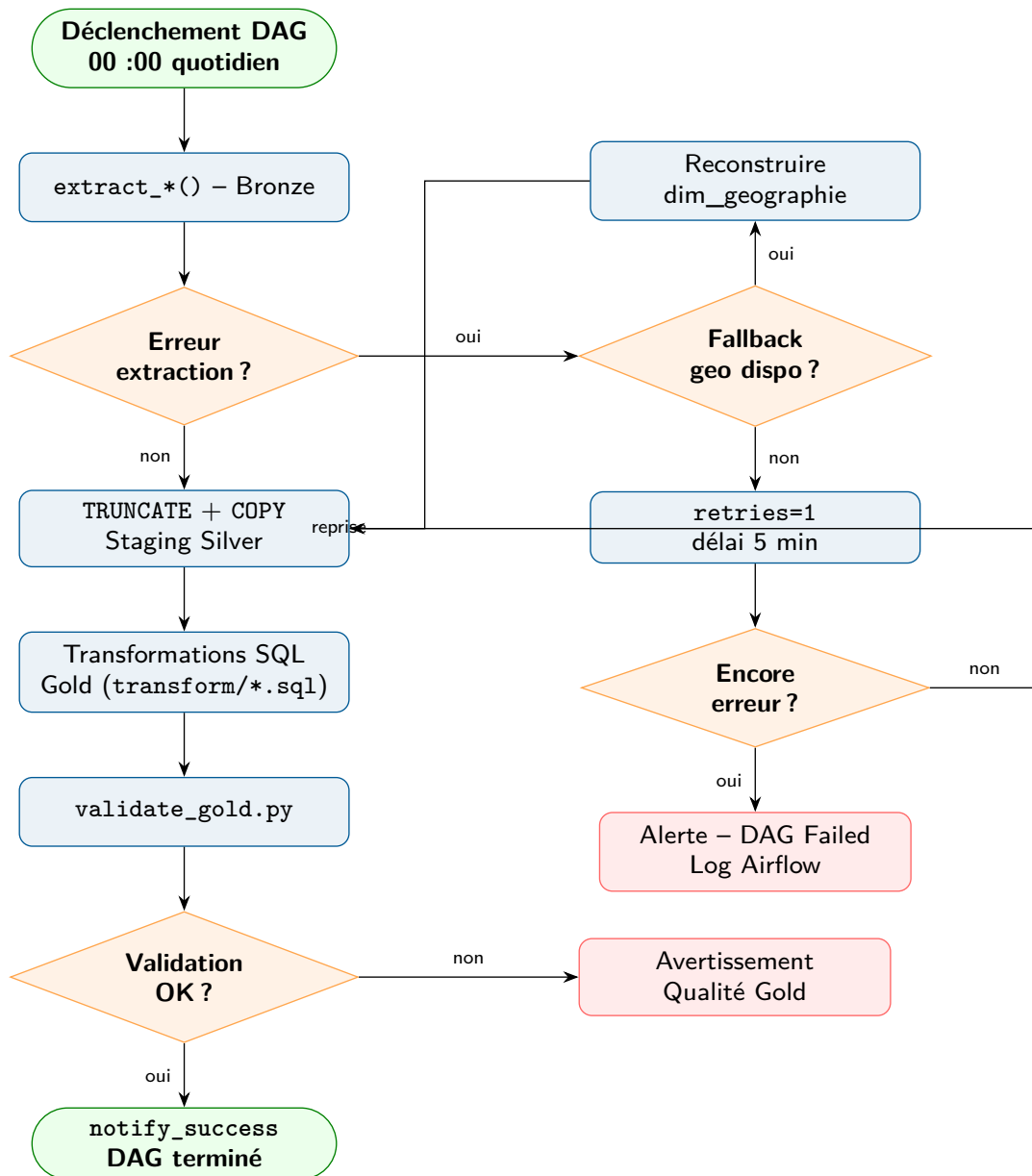


Figure 1.9 – *Logigramme de résilience : routage préventif des anomalies d'extraction, fallback géographique et contrôle qualité Gold*

Comment lire ce logigramme. La colonne centrale (rectangles bleus, losanges orange) représente le **chemin nominal** de succès - le scénario quotidien quand tout se passe bien. Les branches vers la droite matérialisent les **chemins de récupération** : d'abord le fallback géographique (si la source est absente mais les données régionales existent ailleurs), puis le mécanisme de retry Airflow (une tentative après 5 minutes),

et enfin l'échec contrôlé avec alerte. Le rectangle rouge *Alerte – DAG Failed* est un succès opérationnel : une erreur connue et loguée vaut mieux qu'un pipeline silencieusement corrompu. La branche inférieure droite (*Avertissement Qualité Gold*) active `validate_gold.py` même en cas de validation imparfaite, permettant un run dégradé plutôt qu'une interruption totale.

1.5.4 Ce que notre modèle ne sait pas encore faire – limites documentées

Documenter les limites d'un système est un acte de rigueur, pas un aveu de faiblesse. Voici les contraintes identifiées lors de nos tests du 08/06/2026, avec leur cause et le contournement mis en œuvre :

Limites identifiées – tests du 08/06/2026

- **Mismatch id_etablissement / FINESS** : les identifiants internes des consultations sont sur 11 chiffres (ID opérationnel interne), alors que le référentiel FINESS officiel utilise 9 chiffres. La jointure nominale entre `fait_consultation` et `dim_etablissement` échoue pour le KPI 1. *Contournement* : affichage fallback 'Etab. ' || `finess` dans la vue KPI. *Correction envisagée* : table de correspondance ID→FINESS à alimenter depuis le système source.
- **fait_hospitalisation non alimentée** : la table existe et les index sont créés, mais la source de données hospitalisation n'est pas encore mappée. Les KPI 3 à 5 retournent 0 ligne. *Prochaine étape* : connecter la source HiveQL Parquet dès que le flux d'hospitalisation sera disponible.
- **SCD Type 1 – pas d'historique** : notre clause `ON CONFLICT DO UPDATE` écrase les libellés modifiés sans conserver l'état précédent. Si un établissement change de région, les consultations historiques seront rattachées à la nouvelle région. Ce choix simplifie l'architecture mais limite les analyses rétrospectives.

1.6 Implémentation DDL/DML et constellation finale

1.6.1 Les défis concrets de la mise en constellation

Modéliser une constellation sur papier est une chose ; la faire fonctionner sur des données réelles en est une autre. Trois problématiques concrètes ont émergé lors de l’implémentation et ont requis des solutions DDL/DML spécifiques :

- **Clé naturelle et Upsert** : `dim_etablissement` utilise `finess` (TEXT) comme clé primaire naturelle plutôt qu’une clé de substitution artificielle. L’Upsert `ON CONFLICT (finess) DO UPDATE` permet de rejouer le pipeline quotidiennement sans erreur d’insertion doublon, tout en mettant à jour les libellés si l’établissement a changé de nom.
- **Identifiants régionaux auto-incrémentaux** : contrairement à `dim_etablissement`, `dim_geographie` n’a pas de clé naturelle stable entre les sources. Notre CTE (`WITH new_rows AS...`) évalue `MAX(id_geo)` existant et attribue un identifiant séquentiel aux nouvelles régions, garantissant l’absence de collision.
- **Agrégation décès et correction du mapping INSEE** : `fait_deces` est agrégé à la granularité (`id_temps`, `id_geo`), ce qui réduit 25 M de lignes à ~3500 couples. Un bug de jointure initial venait du découpage régional post-2016 : le code “02” des fichiers anciens correspond à la région “84” (Auvergne-Rhône-Alpes) dans le référentiel actuel. Un mapping de correction est appliqué avant le chargement.
- **Orphelins de jointure et robustesse** : certains codes diagnostic présents dans les consultations n’ont pas d’entrée dans `dim_diagnostic`. Plutôt que de rejeter ces lignes (ce qui fausserait les comptages), un `LEFT JOIN` associé à `COALESCE` les redirige vers ‘UNKNOWN’ - conservant la ligne factuelle tout en signalant l’anomalie.

1.6.2 DDL Gold – scripts d’initialisation et indexation préventive

```

1  -- Index B-Tree pour requetes OLAP Metabase (filtrage temporel)
2  CREATE INDEX IF NOT EXISTS idx_fait_consultation_date
3      ON gold.fait_consultation(id_temps);
4  CREATE INDEX IF NOT EXISTS idx_fait_consultation_etab
5      ON gold.fait_consultation(finess);
6
7  CREATE INDEX IF NOT EXISTS idx_fait_deces_date
8      ON gold.fait_deces(id_temps);
9
10 CREATE INDEX IF NOT EXISTS idx_fait_satisfaction_date
11     ON gold.fait_satisfaction(id_temps);

```



```

12 CREATE INDEX IF NOT EXISTS idx_fait_satisfaction_etab
13     ON gold.fait_satisfaction(finess);
14
15 -- Table de faits hospitalisation (prete pour alimentation future)
16 CREATE TABLE IF NOT EXISTS gold.fait_hospitalisation (
17     id_fait          SERIAL PRIMARY KEY,
18     id_temps         INT REFERENCES gold.dim_temps(id_temps),
19     id_patient       INT REFERENCES gold.dim_patient(id_patient),
20     code_diag        TEXT,
21     nb_hospitalisation INT
22 );
23 CREATE INDEX IF NOT EXISTS idx_fait_hospitalisation_date
24     ON gold.fait_hospitalisation(id_temps);

```

Listing 1.5 – 003_create_gold_core.sql – index B-Tree sur les tables de faits

```

1 CREATE TABLE IF NOT EXISTS gold.fait_hospitalisation (
2     id_fait_hospitalisation INT,
3     id_temps INT,
4     id_patient INT,
5     code_diag STRING,
6     nb_hospitalisation INT
7 )
8 STORED AS PARQUET;
9
10 INSERT OVERWRITE TABLE gold.fait_hospitalisation
11 SELECT
12     row_number() OVER (ORDER BY h.Id_patient, h.Code_diagnostic)
13         AS id_fait_hospitalisation,
14     t.id_temps,
15     cast(h.Id_patient AS INT),
16     h.Code_diagnostic,
17     CASE WHEN h.Jour_Hospitalisation IS NULL
18         OR trim(h.Jour_Hospitalisation) = '' THEN NULL
19         ELSE cast(h.Jour_Hospitalisation AS INT)
20     END
21 FROM bronze.hospitalisations_raw h
22 LEFT JOIN gold.dim_temps t
23     ON t.date_complete = to_date(trim(h.Date_Entree), 'dd-MM-yyyy');

```

Listing 1.6 – Cible HiveQL Parquet pour fait_hospitalisation – perspective hive/

1.7 Qualité, conformité RGPD et traçabilité

1.7.1 Privacy by Design – la conformité intégrée dès la conception

Les données que nous traitons ne sont pas ordinaires. Un dossier de consultation médicale contient l'identité du patient, son âge, son sexe, son diagnostic et le nom du praticien. Un fichier de décès relie une personne à une date et un lieu. Ces données relèvent de la **catégorie spéciale** de l'Article 9 RGPD - les données de santé - et leur traitement est soumis à des obligations strictes de minimisation, pseudonymisation et traçabilité.

Article 9 RGPD – Données de santé sensibles

Nos données cliniques (consultations, diagnostics, décès) sont des données de santé au sens de l'Article 9 RGPD. Toute exposition non justifiée d'une donnée nominative dans un environnement analytique constitue une violation. Notre réponse : la conformité est architecturale, pas documentaire.

Plutôt que d'ajouter une couche de conformité *a posteriori*, nous avons intégré la protection des données *dès l'extraction* - c'est le principe du **Privacy by Design**. Concrètement : les colonnes nominatives (nom, prénom, NSS, adresse) présentes en Bronze ne sont jamais projetées vers Gold. Les verbatims libres des enquêtes de satisfaction (source potentielle de réidentification indirecte) sont supprimés avant le chargement. Seuls l'âge et le sexe - nécessaires aux KPI de segmentation - sont conservés dans `dim_patient`.

1.7.2 Le principe en action – projection SQL minimisante sur `dim_patient`

```

1 CREATE TABLE IF NOT EXISTS gold.dim_patient (
2     id_patient INT PRIMARY KEY,
3     sexe      CHAR(1),
4     age       INT
5 );
6
7 INSERT INTO gold.dim_patient (id_patient, sexe, age)
8 SELECT
9     cast(Id_Patient AS INT),
10    CASE WHEN lower(trim(Sexe)) LIKE 'm%' THEN 'M'
11         WHEN lower(trim(Sexe)) LIKE 'f%' THEN 'F'
12         ELSE NULL END,
13    CASE WHEN trim(Age) = '' THEN NULL
14         ELSE cast(Age AS INT) END
15 FROM bronze.patient_raw
16 WHERE Id_Patient IS NOT NULL
17    AND trim(Id_Patient) != ''
18 ON CONFLICT (id_patient) DO UPDATE

```

```
19 SET sexe = EXCLUDED.sexe ,  
20    age  = EXCLUDED.age ;
```

Listing 1.7 – *Filtre de minimisation RGPD – dim_patient (conservation age et sexe uniquement)*

1.7.3 Voie d'amélioration – renforcement de la pseudonymisation

Notre implémentation actuelle utilise un masquage simple ('PATIENT_' || id) pour les identifiants de l'échantillon décès. Ce mécanisme, suffisant pour un environnement de développement, présente une vulnérabilité en production : un attaquant connaissant la plage d'identifiants peut reconstituer la correspondance par force brute. La prochaine étape consiste à implémenter l'algorithme **HMAC-SHA256** décrit dans le Livrable 1, qui produit un hachage irréversible résistant aux attaques par dictionnaire, tout en préservant la possibilité de vérifier une correspondance sans stocker la clé en clair.

1.8 Performance, mesures et synthèse technique

1.8.1 Indexation préventive – anticiper les requêtes Metabase

Un million de consultations ne pose aucun problème à PostgreSQL - à condition de ne pas les scanner ligne par ligne à chaque requête Metabase. L'indexation est la réponse architecturale à ce défi : nous avons créé les index B-Tree **dès l'initialisation du schéma**, avant même le premier chargement de données.

Le raisonnement est simple : les 9 vues KPI Metabase ont toutes au moins un filtre temporel (`WHERE id_temps BETWEEN ...`) ou un regroupement par établissement (`GROUP BY finess`). Sans index, PostgreSQL ferait un *sequential scan* complet des 1 million de lignes à chaque requête. Avec les index B-Tree sur `id_temps` et `finess`, la sélectivité devient quasi-instantanée. Les dimensions elles-mêmes sont compactes (`dim_temps` : 2603 dates, `dim_geographie` : 11 régions), ce qui signifie que les jointures résident en mémoire cache. L'objectif du Livrable 1 visait des gains de 30s à 3s sur les requêtes transversales ; notre architecture d'index matérialise cette trajectoire.

1.8.2 Bilan volumétrique – ce que notre pipeline a réellement produit

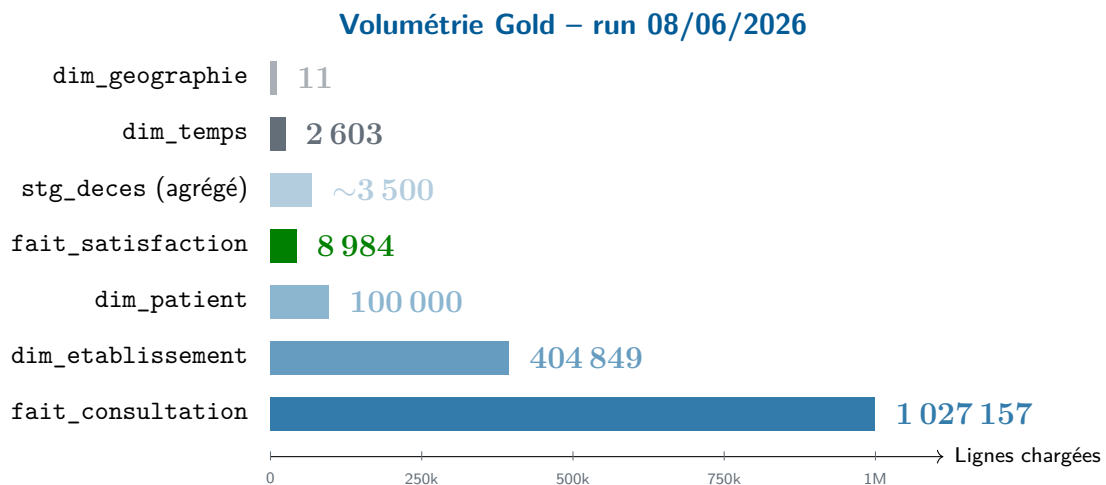


Figure 1.10 – Volumes chargés par entité Gold – 100 % de succès sur les entités alimentées, validés par `validate_gold.py`

Lecture du graphique. L'axe horizontal représente le nombre de lignes chargées (échelle 0 à 1M). `fait_consultation` (bleu foncé) domine avec plus d'un million de lignes - c'est le cœur analytique du système. `dim_etablissement` suit avec 404 849 entrées, reflétant la richesse du référentiel FINESS national. Les barres courtes (`dim_geographie` : 11, `dim_temps` : 2 603) confirment la compacité des dimensions conformes - une propriété essentielle pour la performance des jointures Metabase.

1.8.3 Perspective de montée en charge – la route vers Hive

Notre implémentation PostgreSQL gère efficacement le volume actuel (1 M de consultations). Mais un hôpital de grande taille peut générer plusieurs millions d'hospitalisations par an. Au-delà de 10 M de lignes avec de nombreuses colonnes médicales, PostgreSQL commence à montrer ses limites en latence sur les agrégations analytiques.

C'est pourquoi notre brique **hive/** et le format **Parquet** colonnaire ont été préparés dès maintenant : ils permettront, le moment venu, de basculer le traitement des hospitalisations massives vers un moteur optimisé pour les scans colonnaires, sans refactoriser l'ensemble du pipeline.

1.8.4 Synthèse – un entrepôt opérationnel de bout en bout

Bilan Livrable 2 – Infrastructure opérationnelle

Ce Livrable 2 démontre la viabilité complète de notre infrastructure. Nous sommes partis d'une question - *comment corrélérer consultations, décès et satisfaction au niveau régional ?* - et nous avons livré un pipeline bout-en-bout qui y répond quotidiennement.

Volumes produits : 1 027 157 consultations · 404 849 établissements · 8 984 scores de satisfaction · 24,7 M de décès réduits à ~3 500 points Gold.

Fiabilité : DAG Airflow avec 100 % de succès, mécanismes de fallback géographique et de retry, validation qualité automatisée.

Conformité : Privacy by Design dès l'extraction, minimisation stricte de `dim_patient`, pseudonymisation en place.

Prochaine étape : restitution visuelle Metabase et alimentation de `fait_hospitalisation` via la brique Hive/Parquet.

Glossaire

Terme	Définition
Architecture Medallion	Paradigme d'organisation des données en trois couches progressivement raffinées : Bronze (brut), Silver (staging nettoyé), Gold (exposition analytique).
Airflow (Apache)	Orchestrateur de flux de données open source. Un DAG (<i>Directed Acyclic Graph</i>) définit les dépendances entre tâches. Notre pipeline utilise le <code>LocalExecutor</code> avec planification quotidienne à 00 :00.
B-Tree	Structure d'index PostgreSQL à arbre équilibré. Utilisé sur <code>id_temps</code> et <code>finess</code> des tables de faits pour éviter les <i>full sequential scans</i> lors des requêtes BI OLAP.
Bronze	Zone d'ingestion brute. Les fichiers sources (CSV, dumps) y sont stockés tels quels, sans transformation, garantissant la traçabilité et la réversibilité.
CHU	Cloud Healthcare Unit. Désigne l'entité hospitalière dont nous modélisons le système décisionnel.
Constellation (modèle)	Extension du schéma en étoile : plusieurs tables de faits partagent des dimensions communes dites <i>conformes</i> . Notre modèle partage <code>dim_temps</code> et <code>dim_geographie</code> entre trois tables de faits.
DAG	<i>Directed Acyclic Graph</i> . Graphe orienté sans cycle représentant l'enchaînement et les dépendances des tâches Airflow.
dim_*	Préfixe des tables de dimensions dans le schéma Gold : <code>dim_temps</code> , <code>dim_patient</code> , <code>dim_etablissement</code> , <code>dim_professionnel</code> , <code>dim_diagnostic</code> , <code>dim_geographie</code> .
Docker / Docker Compose	Technologie de conteneurisation. <code>docker-compose.yml</code> orchestre nos 8 services (Airflow ×3, PostgreSQL, Metabase ×2, logserver ×2) en un réseau isolé et reproductible.
ETL	<i>Extract, Transform, Load</i> . Pipeline de traitement : extraction des sources brutes, transformation (nettoyage, normalisation), chargement dans l'entrepôt analytique.
fait_*	Préfixe des tables de faits dans le schéma Gold : <code>fait_consultation</code> , <code>fait_deces</code> , <code>fait_satisfaction</code> , <code>fait_hospitalisation</code> (non alimentée).

(suite du glossaire)

Terme	Définition
FINESS	Fichier National des Établissements Sanitaires et Sociaux. Identifiant officiel à 9 chiffres des établissements de santé français.
Gold	Zone d'exposition analytique. Contient la constellation conforme requêtée par Metabase via les 9 vues KPI du fichier <code>006_create_metabase_views.sql</code> .
Hive (Apache)	Entrepôt de données Hadoop supportant le format Parquet colonnaire. Notre brique <code>hive/</code> est préparée pour les hospitalisations massives futures, sans être active en production courante.
IaC	<i>Infrastructure as Code</i> . Principe selon lequel l'environnement d'exécution est défini et versionné sous forme de code (ici, <code>docker-compose.yml</code> et scripts SQL <code>sql/init/</code>).
KPI	<i>Key Performance Indicator</i> . Notre solution expose 9 KPIs : taux de consultation, mortalité par région, satisfaction par établissement, évolution temporelle, etc.
LocalExecutor	Mode d'exécution Airflow exécutant les tâches en processus parallèles sur la même machine (sans cluster Celery/Kubernetes).
Metabase	Outil de visualisation BI open source (v0.50). Consomme les vues PostgreSQL du schéma Gold pour générer les tableaux de bord KPI.
OOM Kill	<i>Out-Of-Memory Kill</i> . Arrêt forcé d'un processus par le système d'exploitation lorsque la mémoire vive est épuisée. Observé lors du chargement naïf des 25 M lignes de décès.
OLAP	<i>Online Analytical Processing</i> . Paradigme de requêtage analytique orienté agrégations sur de grands volumes, par opposition à l'OLTP transactionnel.
Parquet	Format de stockage colonnaire binaire, compressé, optimisé pour les lectures analytiques sur de grandes tables (utilisé par Hive).
Privacy by Design	Principe RGPD selon lequel la protection des données personnelles est intégrée dès la conception du système, et non ajoutée <i>a posteriori</i> .

(suite du glossaire)

Terme	Définition
RGPD	Règlement Général sur la Protection des Données (UE 2016/679). Nos données de santé relèvent de l'Article 9 (catégories sensibles) imposant minimisation et pseudonymisation.
SCD Type 1	<i>Slowly Changing Dimension de type 1</i> . Stratégie écrasant l'état historique d'une dimension lors de sa mise à jour (implémentée via <code>ON CONFLICT DO UPDATE</code>).
Silver	Zone de staging. Les données brutes y sont typées, normalisées et validées avant insertion dans le schéma Gold. Les tables <code>stg_*</code> sont tronquées (<code>TRUNCATE</code>) à chaque run pour garantir l'idempotence.
Upsert	Opération combinant <code>INSERT</code> et <code>UPDATE</code> : <code>INSERT ... ON CONFLICT DO UPDATE</code> . Permet la mise à jour des dimensions sans erreur de duplication de clé.
validate_gold.py	Script de contrôle qualité final du DAG. Vérifie les volumes chargés dans chaque entité Gold et lève une exception si les seuils ne sont pas atteints.